|Dullani | دُلني

Smart Parking detection with proximity recommendations

Student Name	Student ID	Section	Email	Role
Noran Abdullah Aljodi	445005178	3	s445005178@uqu.edu.sa	Leader
Reema Salem Alharbi	445000219	3	s445000219@uqu.edu.sa	Member
Enas Abdullah Saud	44512190	3	s44512190@uqu.edu.sa	Member
Asail Hamdan Al Osaimi	445000051	3	s445000051@uqu.edu.sa	Member
Rema Saed Althaqfi	445005754	3	s445005754@uqu.edu.sa	Member

AI Systems Design AI3104
Umm Al Qura University

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Abstract

The increasing difficulty of finding available parking spaces in crowded environments has created a need for intelligent parking management solutions. This project proposes a smart parking system that combines computer vision and path-finding techniques to automatically detect parking occupancy and recommend the nearest available parking space. The system utilizes the **YOLOv8** object detection model to classify parking spaces as occupied or available, while the **A* algorithm** is used to generate an efficient route to the nearest vacant spot. The proposed solution aims to reduce parking search time, minimize traffic congestion within parking areas, and improve the overall user experience. Experimental results demonstrated strong detection performance, achieving high **Precision**, **Recall**, and **mAP** scores, indicating the effectiveness of the proposed system in supporting smart parking management.

- **The complete project, including source code, trained model, and deployment files, is publicly available on GitHub at:https://github.com/RemaAlthagafi/smart-parking-detection**

Chapter 2: AI Lifecycle

2.1 Introduction & Problem Definition

2.1.1 Background

Finding an available parking space has become a significant challenge in crowded environments such as universities, hospitals, shopping malls, and public parking facilities. Drivers often spend considerable time searching for vacant parking spaces, which leads to increased traffic congestion, fuel consumption, and overall inconvenience. As the number of vehicles continues to grow, traditional parking management approaches are becoming less effective in meeting real-world parking demands.

Recent advances in computer vision technologies have enabled the development of intelligent systems capable of automatically analyzing parking environments and detecting parking occupancy. Such systems can help drivers quickly identify available parking spaces and reduce unnecessary vehicle

movement within parking areas. Therefore, smart parking solutions have emerged as a practical approach to improving parking efficiency and enhancing the overall user experience.

2.1.2 Problem Statement

Some people face difficulty finding a parking space in the shortest possible time, especially in crowded places such as malls and hospitals. Therefore, there is a need for an automated system capable of detecting the status of the nearest available parking space and enhancing the user's experience.

2.1.3 Project Overview

Our project focuses on developing a smart parking system that utilizes computer vision and path-finding algorithms to improve parking efficiency in crowded environments. Therefore, the system is designed to automatically detect available parking spaces from visual data and guide drivers to the nearest empty spot. We aim to reduce the time spent searching for parking, decrease traffic congestion, and improve the overall driving experience.

2.1.4 Project Objectives

1. Detect and classify parking spaces as occupied or available.
2. Identify the nearest available parking space based on the user’s location.
3. Guide drivers using the A* algorithm to find the optimal path.
4. Improve efficiency and reduce the time spent searching for parking spaces in crowded areas.

2.1.5 Target Users

The main users of this system are **drivers** who are looking for available parking spaces in crowded places such as universities, shopping malls, hospitals, and public parking areas. Many drivers spend a long time searching for empty parking spaces, especially during busy times. This causes delays, traffic congestion, and extra fuel consumption. The proposed system helps drivers by automatically detecting which parking spaces are available or occupied using computer vision. The system then identifies the nearest available parking space, which helps reduce the time needed to find parking and makes the

parking process easier and faster. **Other users** of the system may include **parking managers** or **building administrators**. They can benefit from understanding how parking spaces are used and how often they are occupied. This information can help improve parking organization and management.

2.1.6 Project Scope

The scope of this project includes the development of a smart parking system that detects parking occupancy using the YOLOv8 object detection model and recommends the nearest available parking space using the A* path-finding algorithm. The system processes parking images, classifies parking spaces as occupied or available, and generates a visual route from the user's location to the selected parking space through a user interface.

The project focuses on image-based parking analysis and recommendation within parking environments. Features such as live camera integration, GPS navigation, and mobile application deployment are considered outside the current project scope and are proposed as future enhancements.

2.1.7 Feasibility and Risk Assessment

The project was considered technically feasible because it relied on standard computer vision techniques and publicly available parking datasets. The use of the PKLot dataset and the YOLOv8 framework enabled the successful development and evaluation of the proposed system. The dataset contains labeled parking images captured under different lighting and weather conditions, making it suitable for realistic parking occupancy detection tasks.

Several risks were identified during the development phase, including lighting variations, image quality issues, occlusion, camera angle differences, and dataset imbalance, all of which could affect detection accuracy and recommendation quality. Additional challenges included computational resource limitations and training requirements. These risks were addressed through dataset preprocessing, image standardization, careful model configuration, and the use of suitable training parameters to improve overall system reliability.

2.1.8 Ethical, Legal, Societal Considerations

1. Ethical Considerations

The proposed AI-based parking system must adhere to fundamental ethical principles to ensure responsible and trustworthy deployment. One of the primary concerns is user privacy, particularly when sensors or camera-based technologies are used to detect parking availability. The system should implement data minimization strategies and avoid collecting personally identifiable information (PII) unless strictly necessary, in accordance with the principles established by Saudi Data and Artificial Intelligence Authority (SDAIA) (SDAIA, 2023). In addition, fairness and non-discrimination must be ensured. The system should provide equal access and service quality to all users without bias. This requires training AI models on diverse and representative datasets to mitigate potential bias and ensure equitable outcomes. Transparency and accountability are also critical. Users should be clearly informed about how the system operates, how decisions are made, and how their data is collected and used (OECD, 2021).

2. Legal Considerations

From a legal standpoint, the system must comply with data protection laws and regulations in Saudi Arabia, particularly the Personal Data Protection Law (PDPL), which governs the collection, processing, and storage of personal data (SDAIA, 2023). If the system relies on surveillance technologies such as cameras, it is essential to obtain the necessary approvals from relevant authorities and ensure compliance with local regulations concerning public monitoring and data usage. Moreover, all external resources, including datasets, APIs, and software libraries, must be properly licensed and referenced to avoid violations of intellectual property rights.

3. Societal Considerations

The proposed system is expected to deliver significant societal benefits, including reducing traffic congestion, minimizing time spent searching for parking, and enhancing overall user experience. These benefits align with the objectives of Saudi Vision 2030, particularly in promoting smart city initiatives and digital transformation. However, potential challenges must also be considered. One such challenge is over-reliance on automated systems, where users may depend entirely on AI-driven decisions.

Additionally, accessibility should be ensured so that the system can be effectively used by individuals with varying levels of technical expertise.

2.1.9Assumptions and Constraints

Assumptions

- Users have access to smartphones or digital platforms.
- Parking data is updated accurately and in real time.
- Sensors and data sources provide reliable information.

Constraints

- Limited availability or quality of data.
- Hardware and infrastructure limitations.
- Legal restrictions on data collection (e.g., camera usage).
- Time and resource constraints during development.

2.2 Data Collection and Preparation

2.2.1 Dataset Source and Justification

The dataset selected for this project is a pre-processed version of the PKLot dataset available on Kaggle. It is derived from the original PKLot research dataset and adapted to support object detection tasks. The dataset includes 12,416 images of outdoor parking lots captured by surveillance cameras in three separate locations in Brazil: PUCPR, UFPR04, and UFPR05. The images capture various weather conditions, including sunny, cloudy, and rainy days, providing a robust environment for evaluating computer vision models under different lighting and shadow effects. This variation in lighting and weather conditions helps prevent the model from overfitting to a specific pattern, enabling it to accurately recognize parking spaces across various times and real-world environmental conditions. Each parking space within an image is manually labeled as "Occupied" or "Empty." The images are in JPEG format, with a resolution of 1280×720. The original annotations have been converted into axis-aligned

bounding boxes, which are available in CSV and TXT formats, ensuring that the dataset is fully compatible with standard object detection frameworks.

2.2.2 Data Preparation and Preprocessing

The preprocessing stage is responsible for preparing and organizing the parking images before they are passed to the YOLOv8 model for training and inference. Proper preprocessing is essential to ensure dataset consistency and support efficient model learning. The data set utilized in this project is primarily derived from the **PKLot** parking data set. To reduce the time and effort required for manual annotation, a pre-labeled version of the same dataset was obtained from **Roboflow Universe**. This version includes ready-made annotations for parking-related objects and vehicles using bounding boxes in YOLO format.
The preprocessing pipeline includes the following steps:

- **Dataset Organization:** The dataset is organized into training, validation, and testing subsets to support model training and evaluation.
- **Data Cleaning:** The dataset was prepared using a pre-labeled version obtained from **Roboflow**, ensuring compatibility with the **YOLOv8** training pipeline.
- **Image Resizing:** All images are automatically **resized to 640×640** pixels to ensure compatibility with the YOLOv8 architecture and maintain consistent input dimensions across the dataset.
- **Label Verification and Preparation:** Bounding box annotations provided in YOLO format were verified to ensure correct object localization and annotation consistency.

The overall objective of preprocessing is to provide clean, standardized, and properly annotated image data that enables accurate and efficient parking occupancy detection using YOLOv8.

2.2.3 Data Governance

The dataset used in this project was obtained from publicly available and properly documented sources, including **Kaggle** and **Roboflow**. The dataset is used solely for academic and research purposes. Since the **PKLot** dataset does not contain personally identifiable information such as faces or license plate

numbers, privacy concerns are minimized. In addition, the dataset was obtained in a pre-labeled format that is compatible with the YOLOv8 framework, which facilitated the training and evaluation process.

2.3 Model Development and Training

2.3.1 Algorithm Selection

The proposed system combines two algorithms to perform parking occupancy detection and nearest parking recommendation.

YOLOv8 Model:

The YOLOv8 model was selected for the parking occupancy detection and classification module. It is used to detect parking spaces within the image of the parking lot and classify each parking space as either occupied or available. YOLOv8 was chosen because it provides high detection accuracy, fast processing speed, and the ability to perform object detection and classification within a single model.

A* Algorithm:

The A* algorithm was selected for the nearest parking recommendation module. After identifying the available parking spaces using YOLOv8, the A* algorithm calculates the shortest path from the user's location to the nearest available parking space. This allows the system to provide efficient parking recommendations and route guidance within the parking environment.

2.3.2 System Architecture

Our system is based on intelligent detection of available parking spaces combined with recommending the nearest parking spot. The system is designed as a sequential series of stages to ensure smooth data flow and allow future development and scalability.

The system begins with the parking data set stage, which contains images or video clips of parking lots captured under different conditions such as variations in lighting and camera angles. After that, the data is loaded into the working environment for processing.

Next, data cleaning is performed. In this stage, corrupted, distorted, duplicated, or low-quality images are removed to prevent negatively affecting the model's performance and to maintain dataset consistency before the training process.

After that, image preprocessing is applied All images are resized to a unified fixed size to ensure consistency across the dataset. The dataset is then split into training, validation, and testing sets. During model training and inference, the YOLO model performs internal processing such as normalization automatically; therefore, no additional manual normalization is applied in the preprocessing stage.

Then, the parking space detection stage is performed. In this stage, parking spaces are identified within the image or video using a YOLO object detection model to detect parking spaces and their occupancy status.

After extracting each parking slot individually, the system proceeds to the occupancy classification stage. In this stage, each parking space is analyzed to determine whether it is available or occupied based on the visual information extracted from the image.

Next, the system moves to the decision-making and routing stage. First, only the available parking spaces are filtered from the detected slots. The user's location is mapped directly onto the parking lot image as a visual reference point. To determine the optimal path within the parking layout,

the system applies the A* search algorithm, which is used to compute the shortest and most efficient route from the user's position to the nearest available parking space based on a graph representation of the parking structure. The closest vacant slot is then highlighted visually on the interface as the optimal choice.

After selecting the parking space, the system determines the route within the same visual environment, guiding the user through the parking layout using image-based navigation enhanced by the computed A* path rather than external maps or numerical calculations.

Finally, the results are displayed through the user interface. The parking lot image is shown with occupied and available spaces clearly distinguished, while both the user's position and the nearest available parking space are visually highlighted to provide clear guidance.

Overall, the architectural design allows future scalability, such as integrating live cameras, which makes the system suitable for real-world applications.

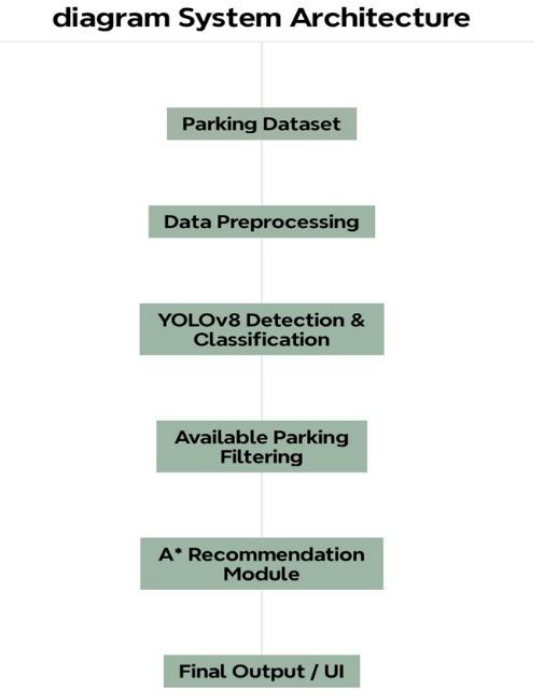

Figure 2.1: Proposed System Architectu

2.3.3 Overall System Workflow

Overall System Workflow The proposed system integrates computer vision techniques with intelligent path-finding algorithms to automate parking management and assist users in finding the nearest available parking space. **The system workflow is divided into two main stages:**

1. **Parking Detection and Classification Stage: The YOLOv8 model** is used to analyze parking lot images, detect parking spaces, and classify each parking space as either occupied or vacant.

2. **Nearest Parking Recommendation Stage: The A* algorithm** uses the detected parking space information to calculate the shortest route from the user's current location to the nearest available parking space.

2.3.4 Model Development Process

The system development process consists of two main stages: parking occupancy detection and nearest parking recommendation.

Image Input and Processing: Parking lot images from the PKLot dataset are provided as input to the system. The images are prepared and resized to match the YOLOv8 input requirements before being processed by the detection model.

Detection and Classification: The YOLOv8 model analyzes the parking lot image, detects parking spaces, and classifies each parking space as either occupied or available. The output consists of bounding boxes and classification labels for each detected parking space.

Path Generation and Recommendation: After detecting the available parking spaces, the system filters only the vacant parking spaces. The parking environment is represented as a two-dimensional grid where accessible areas are treated as traversable nodes and occupied parking spaces are treated as blocked nodes. The user's location is defined as the starting point, while the selected vacant parking space represents the destination.

The **A* path-finding algorithm** is then applied to compute the shortest route between the two points using the following **evaluation function:**

$$f(n) = g(n) + h(n)$$

Where:

- ***g(n):*** Represents the actual path cost from the starting node.
- ***h(n):*** Represents the estimated remaining cost to the destination node.
- ***f(n):*** Represents the total estimated path cost.

Based on the computed path costs, the algorithm selects the most efficient route to the nearest available parking space. The generated path is then visualized on the parking image together with the recommended parking space to provide clear navigation guidance.

Model Testing: After training, the best-performing model was used to perform predictions on unseen test images. A confidence threshold of 0.25 was applied to ensure that only reliable detections were considered. The generated output included parking occupancy predictions and visualized recommendation results.

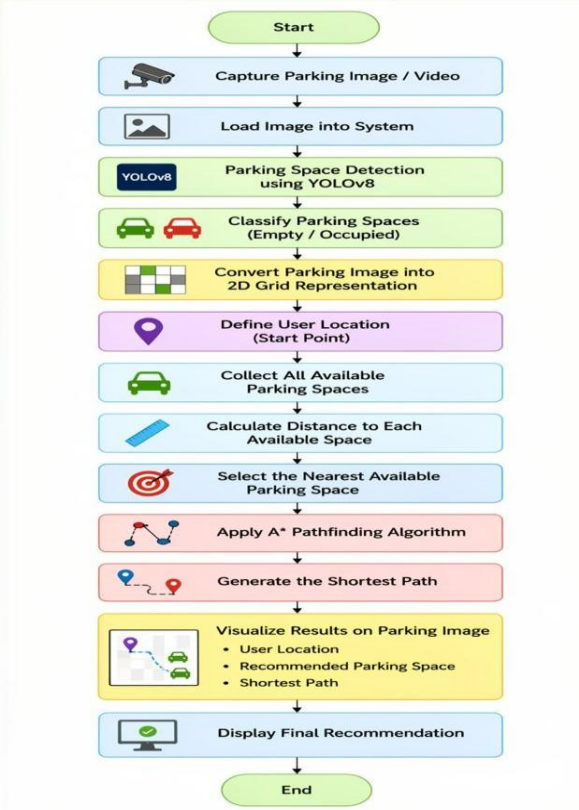

Figure 2.2: the complete workflow of the proposed system

2.3.5 Hyperparameter Configuration

The YOLOv8 model was trained using the following configuration:

- **Epochs = 15**
- **Batch Size = 8**
- **Image Size = 640 × 640**
- **Confidence Threshold = 0.25 (during testing)**

These parameters were selected to provide a balance between **training performance**, **computational efficiency**, and **detection accuracy**.

2.3.6 Evaluation Metrics

The performance of the proposed system was evaluated using standard object detection metrics commonly adopted in computer vision applications. **For the parking occupancy detection module**, the evaluation was based on:

- **Precision** measures how many detected parking spaces are correctly classified.
- **Recall** measures the ability of the model to detect all relevant parking spaces.
- **mAP50** measures the model performance at an Intersection over Union (IoU) threshold of 0.5.
- **mAP50-95** provides a more comprehensive evaluation across multiple IoU thresholds and reflects the overall detection performance of the model.

These metrics were used to assess the effectiveness of the YOLOv8 model in detecting and classifying parking spaces as occupied or available under different parking conditions.

In addition to the parking occupancy detection module, **the nearest parking recommendation module** was evaluated after identifying the available parking spaces using the YOLOv8 model. The A* path-finding algorithm was then applied to generate a route from the user's location to the nearest available parking space. The recommendation module was evaluated through **visual verification** of the generated results, ensuring that the selected parking space was available and that the generated path correctly connected the user's location to the recommended parking space. The output results were displayed directly on the parking image, including the user location, selected parking space, and computed route.

Together, these evaluation methods provide an assessment of both the detection performance and the recommendation functionality of the proposed smart parking system.

2.4 Deployment and Serving

2.4.1 Pipeline Validation

Before deploying the system as a web application, the integrated pipeline was validated to ensure correct communication between the parking occupancy detection module and the parking recommendation module.

Detection Validation: The trained **YOLOv8** model was tested to ensure that parking spaces were correctly detected and classified as either Occupied or Empty.

Data Conversion Validation: The detection outputs were converted into a grid representation suitable for the recommendation module. Occupied parking spaces were treated as blocked locations, while available parking spaces were considered valid destinations.

Route Validation: The **A* algorithm** was tested to verify that it correctly generated the shortest route from the user's location to the nearest available parking space.

The validation process confirmed the successful integration of the detection and recommendation stages before deployment.

2.4.2 Web Serving and Deployment Framework

To provide an accessible and interactive environment for users, the proposed system was deployed as a web application using **Gradio** and **Hugging Face Spaces**.

Gradio Interface: Gradio was used to develop the web-based user interface, allowing users to upload parking lot images and view the generated results directly through the browser.

Hugging Face Spaces: The complete application, including the trained **YOLOv8** model, required dependencies, and processing logic, was deployed on Hugging Face Spaces.

System Execution Workflow: When a user uploads a parking image, the **YOLOv8** model detects and classifies parking spaces as occupied or available. The **A* algorithm** then identifies the nearest available parking space and generates the shortest route from the user's location. The final result is displayed through the web interface, showing parking occupancy information and the recommended parking path.

2.5 Monitoring and Maintenance

2.5.1 Monitoring Strategy

The proposed system includes basic monitoring mechanisms to observe the performance of both the parking occupancy detection module and the parking recommendation module. Model outputs, parking occupancy predictions, and generated recommendations can be reviewed during system operation to verify correct functionality. In addition, the web application environment allows monitoring of system execution and identification of potential processing issues during inference.

2.5.2 Scalability and Update Strategy

The proposed system is designed with a modular architecture, which allows future expansion and enhancement without requiring significant modifications to the core system components. The separation between the parking occupancy detection module and the parking recommendation module improves system scalability and simplifies future development.

The system can be extended to support larger parking environments by processing additional parking layouts and datasets. Future updates may also include integrating live camera feeds to enable real-time parking monitoring, developing mobile application support, and incorporating GPS-based navigation services.

In addition, the YOLOv8 model can be updated and retrained using newly collected parking images captured under different environmental and weather conditions. This allows the system to continuously improve its detection performance and adapt to new parking scenarios. The modular design also enables individual components to be updated independently while maintaining the overall functionality of the system.

2.5.3 Reliability Considerations

The proposed system is designed using a sequential pipeline architecture that separates the parking occupancy detection module from the parking recommendation module. This modular structure improves system reliability by allowing each component to perform a specific task independently while maintaining smooth data flow between stages.

The YOLOv8 model is responsible for detecting and classifying parking spaces as occupied or available, while the A* algorithm utilizes the detection results to generate the shortest route to the nearest available parking space. This separation simplifies testing, maintenance, and troubleshooting, reducing the likelihood of system failures affecting the entire workflow.

In addition, the system was trained and evaluated using the PKLot dataset, which contains parking images captured under different lighting and weather conditions. This diversity helps improve the model's generalization capability and supports more reliable performance across various parking environments.

Furthermore, the use of standardized preprocessing steps and consistent input formatting contributes to stable system behavior and reliable prediction outputs during operation.

2.5.4 Fault-Tolerance Considerations

To improve system reliability and maintain stable performance, several fault tolerance considerations are incorporated into the proposed system. The system is designed to handle issues such as unreadable images, missing labels, or invalid inputs by skipping corrupted data during preprocessing to prevent interruptions during training and inference. In addition, using a pre-labeled and organized dataset helps reduce annotation inconsistencies and minimizes detection errors. The dataset is also divided into training, validation, and testing subsets to monitor model behavior and reduce the risk of overfitting or unreliable predictions. Furthermore, YOLOv8 provides confidence scores for detections, allowing low-confidence predictions to be filtered in order to improve output reliability and reduce false detections during real-time operations.

2.5.5 Security Considerations

A significant advantage of selecting the PKLot dataset is its inherent contribution to system security and privacy; since the images lack sensitive personal identifiers such as discernible faces or license plates the risk of data leakage or privacy breaches is mitigated at the source [4]. This strategic decision shifts the security focus to Model Integrity, a key component of AI security that protects the system's logic from tampering. Specifically, robust data validation during the cleaning phase serves as a defense against Data Poisoning, ensuring that no malicious or degraded inputs compromise the model's training or lead to misclassification. Furthermore, the system reinforces security by encrypting all communications and user location data during transmission, aligning with best practices for securing AI lifecycles, and maintaining a resilient security posture.

2.5.6 Disaster Recovery Considerations

To support system recovery and maintainability, the trained YOLOv8 model weights, source code, and deployment files are stored separately and managed through version-controlled repositories. This allows the system to be restored and redeployed if necessary.

In addition, the deployment environment on Hugging Face Spaces enables the application to be rebuilt and relaunched using the stored project files and dependency configurations. This ensures that the system can be recovered and returned to operation without requiring the model to be retrained from the beginning.

Chapter 3: Result

3.1 Detection Results

Metric	Result
Precision	99.7%
Recall	99.7%
mAP50	99.4%
mAP50-95	95.1%

Table 3.1 Detection Performance

The **YOLOv8 model** achieved high performance across all evaluation metrics. The results obtained indicate that the model was able to accurately detect and classify parking spaces as occupied or available under different parking conditions.

Figure 3.1: Detection results under different scenarios

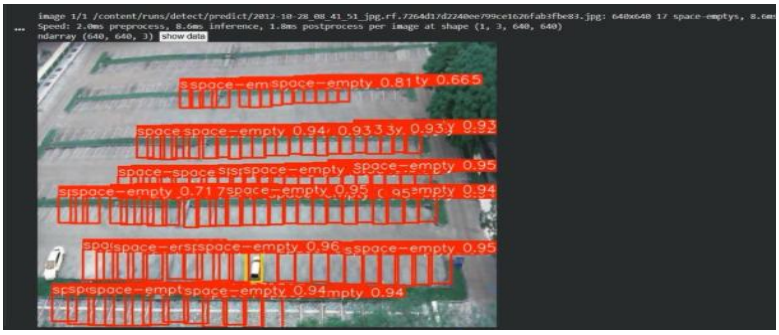

Figure 3.2: Detection results showing empty parking spaces

Figure 3.3: Detection of empty and occupied parking spaces.

3.2 Recommendation Results

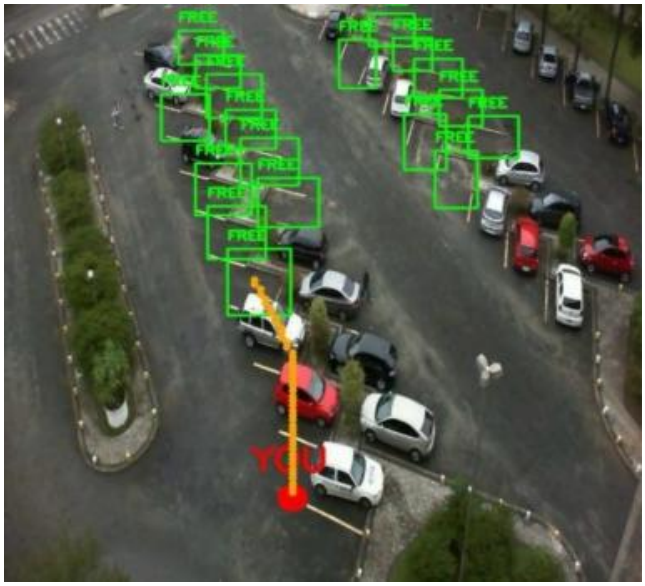

Figure 3.4: shortest path to the nearest available parking space

The recommendation module successfully identified the nearest available parking space and generated the shortest route from the user's location using the **A* algorithm**. The generated route avoided occupied parking spaces and provided a direct path to the recommended destination.

3.3 Web Deployment Results

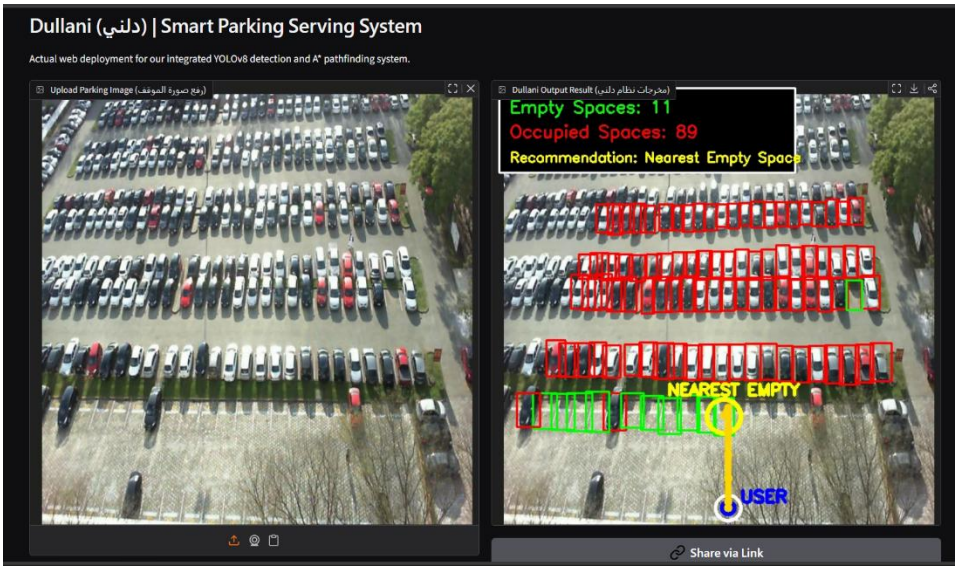

Figure 3.5: Web Deployment of the Dullani Smart Parking System

The complete system was successfully deployed as a web application using **Gradio** and **Hugging Face** Spaces. Users can upload parking lot images and receive parking occupancy detection results together with parking recommendations through an interactive interface.

3.4 Performance Analysis

The experimental results demonstrate the effectiveness of the proposed smart parking system in both parking occupancy detection and parking recommendation. The YOLOv8 model achieved high detection performance and successfully distinguished between occupied and available parking spaces with accurate localization results.

The parking occupancy detection module produced reliable predictions across different parking images, allowing the system to accurately identify available parking spaces for further processing. The obtained **Precision**, **Recall**, **mAP50**, and **mAP50-95** results indicate that the model achieved strong detection performance and effective parking space classification. High Precision values indicate a low number of false positive detections, while high Recall values demonstrate the model's ability to successfully

identify most parking spaces within the parking environment. Furthermore, the high mAP50 and mAP50-95 scores reflect accurate localization and classification performance, confirming the effectiveness of the YOLOv8 model in parking occupancy detection.

In addition, the integration between the YOLOv8 detection module and the A* recommendation module enabled the system to successfully generate parking recommendations and route guidance. **Visual evaluation** confirmed that the system consistently identified the nearest available parking space and generated valid routes from the user's location to the recommended destination. The A* algorithm effectively computed the shortest path to the selected parking space, demonstrating the system's ability to provide accurate parking recommendations and route guidance.

Overall, the results demonstrate that the proposed system successfully combines computer vision and path-finding techniques to support intelligent parking management and improve the parking search process.

3.5 Interactive User Interface

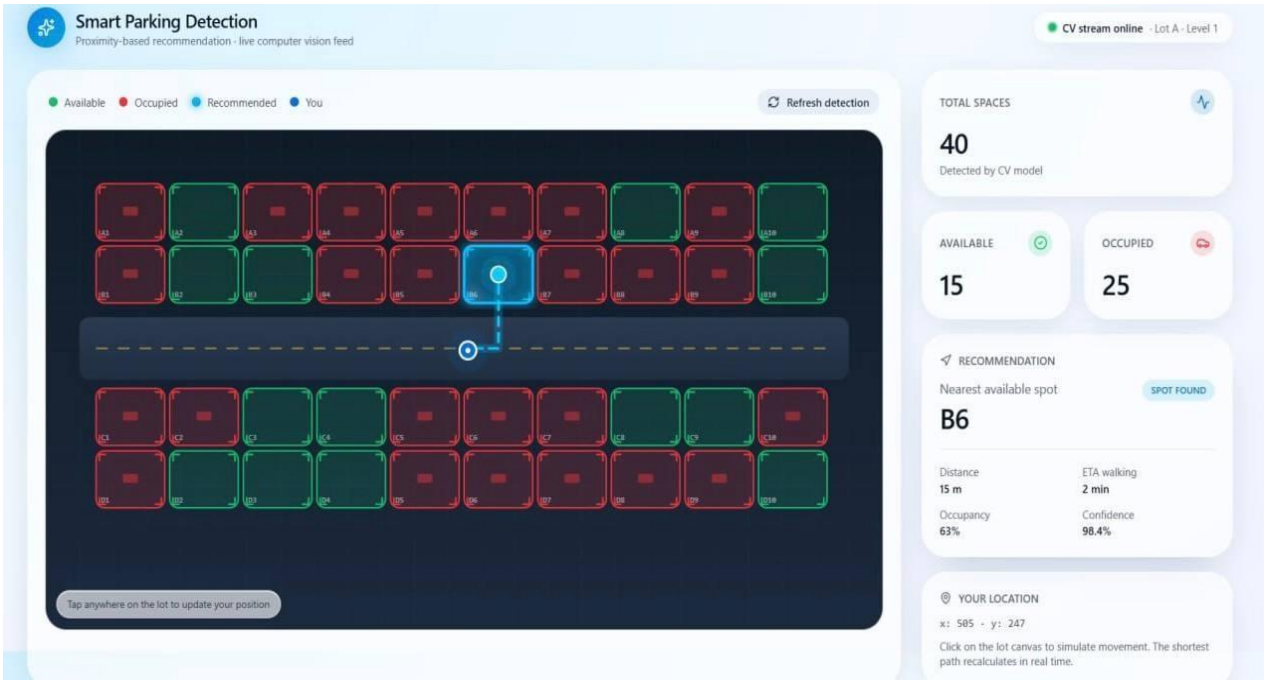

Figure 3.6: Interactive User Interface for Smart Parking Detection and Nearest Parking Recommendation

The developed user interface provides an interactive visualization of the proposed smart parking system. The interface displays parking spaces with different colors representing their status, where green indicates available spaces and red indicates occupied spaces. In addition, the user's current location is represented within the parking layout, while the recommended nearest parking space is highlighted separately. The system also visualizes the shortest calculated route between the user's location and the recommended parking space using the A* algorithm.

The interface supports dynamic interaction by allowing the user position to be changed within the parking layout. Whenever the user location changes or the detection process is refreshed, the system automatically recalculates the nearest available parking space and updates the displayed route accordingly. Furthermore, the interface includes statistical information such as the total number of parking spaces, available spaces, occupied spaces, and recommendation details to improve usability and system visualization.

3.6 Testing, Deployment and System Validation

3.6.1 Monitoring and Logging Hooks

To improve system observability during deployment, a monitoring and logging mechanism was integrated into the deployed application. The monitoring component provides real-time information about the parking detection and recommendation process, allowing the system status to be tracked during inference.

The monitoring framework records several operational metrics, including the total number of detected parking spaces, the number of available and occupied spaces, model inference confidence, and overall execution latency. These metrics are displayed through the user interface and logged during runtime to support performance monitoring and system validation.

Figure 3.7 illustrates a successful inference scenario in which the system **detected 102 parking spaces**, including **28 available spaces and 74 occupied spaces**. The monitoring logs reported an average model

confidence of 0.90 and a processing **latency of approximately 3.39 seconds**, indicating stable detection performance and successful completion of the processing pipeline.

Figure 3.8 presents a more challenging scenario using an image outside the training dataset. In this case, the model detected only one occupied object with a **lower confidence score of 0.55** and **no available parking spaces**. Although the recommendation module could not generate a parking recommendation due to the absence of available parking spaces detected, the monitoring system successfully recorded all execution details, and the application completed the inference process without errors or interruptions.

The collected logs support debugging, performance evaluation, and system validation by providing visibility into the behavior of the deployed system during runtime. Furthermore, they help identify unusual prediction patterns, low-confidence detections, and potential system issues that may require further improvement.

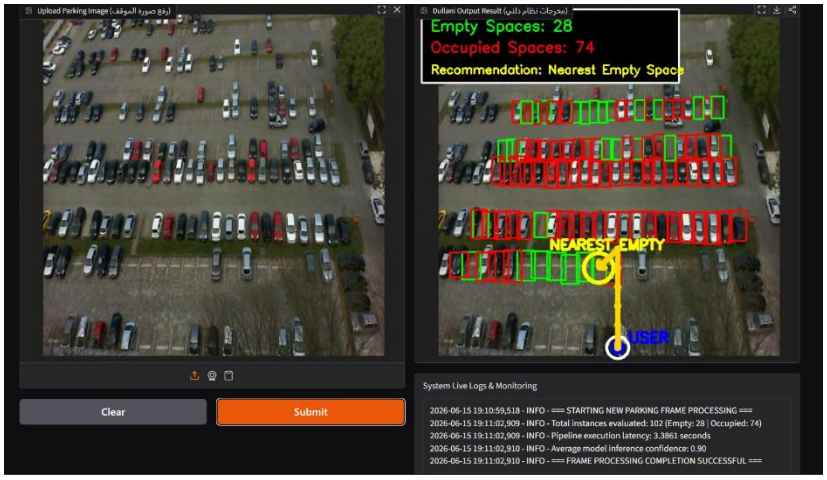

Figure 3.7: Successful System Monitoring

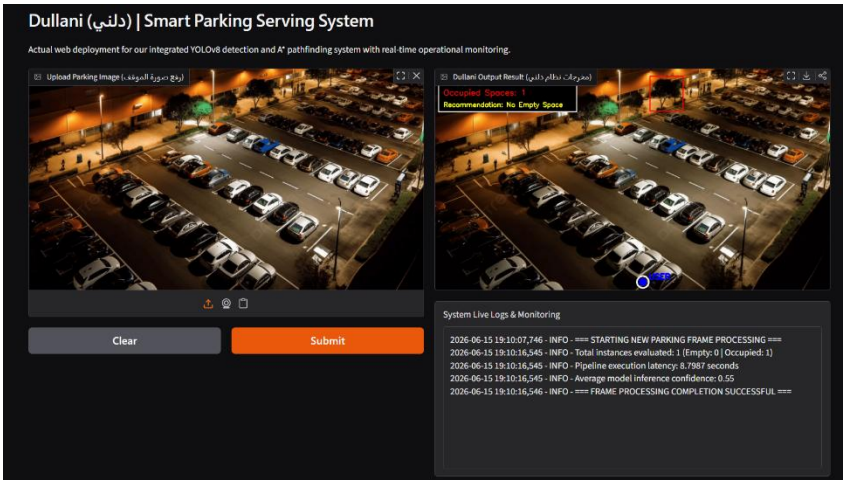

Figure 3.8: System Response to an Unseen Input Image

3.6.2 CI/CD Integration Plan or Scripts

Since model training and experimentation are carried out on Google Colab, a fully automated production-level CI/CD pipeline is not required. Instead, a semi-automated workflow is defined with clear stages to ensure consistency and reliability across training, evaluation, and deployment to the Hugging Face Space.

The workflow is divided into two phases. The CI phase begins with updating the notebook cells, followed by validating the dataset labels and image paths before training. The YOLOv8 model is then trained using the predefined training hyperparameters, and performance is evaluated using model.val () to compute mAP, Precision, and Recall. A quality gate is applied at this stage—if mAP falls below 0.90, training is re-run with adjusted hyperparameters such as increased epochs, modified augmentation settings, or a different confidence threshold.

The CD phase is triggered only after the quality gate is passed. The best-performing model (best.pt) is saved to Google Drive, then loaded and tested on unseen images. The A* path-finding output is verified through visual inspection within the Colab notebook. Once validated, the model file is pushed to the project's Hugging Face Space repository, where the Gradio interface automatically reloads it, making the updated model available through the live web application.

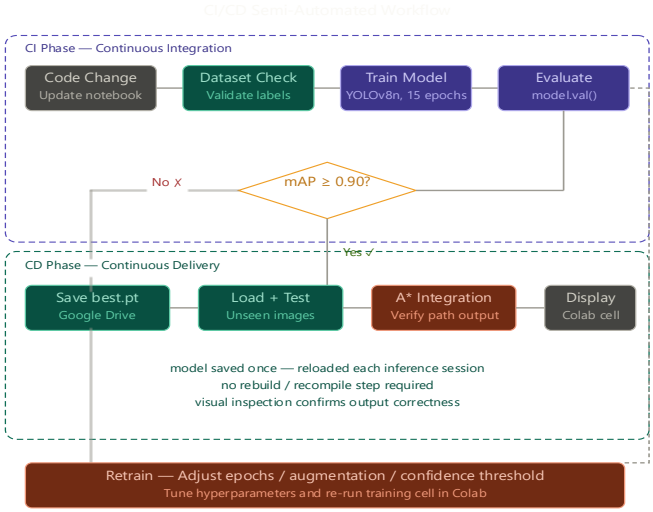

Figure 3.9: CI/CD Workflow

3.6.3 Error Handling and Testing Coverage

```
assert best_path is not None and len(best_path) > 0, '
print(f"A* path validation passed - {len(best_path)} s

print("All tests passed successfully")
```

```
Image validation passed
Model validation passed
Detection results validation passed
A* path validation passed - 21 steps to nearest space
All tests passed successfully
```

```
print(f"mAP50: {map50:.4f} {'PASSED' if map50 >= 0.90 else 'FAILED'}")

print("All tests passed successfully")
```

```
Dataset validation passed
Model validation passed
Ultralytics 8.4.58 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 2565.0±797.4 MB/s, size: 70.1 KB)
val: Scanning /content/datasets/valid/labels.cache... 2483 images, 59 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 2483/2483 1.0Git/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 3% ──────────── 5/156 2.3s/it 11.8s<5:51WARNING ⚠
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 156/156 3.6it/s 43.6s
                   all       2483     143316      0.997      0.992      0.994      0.949
           space-empty       2062      73629      0.998      0.994      0.994      0.961
         space-occupied       1967      69687      0.996      0.989      0.994      0.937
Speed: 1.3ms preprocess, 4.1ms inference, 0.0ms loss, 3.3ms postprocess per image
Results saved to /content/runs/detect/val-2
mAP50: 0.9942 PASSED
All tests passed successfully
```

Figure 3.10: Error handling and testing coverage results in the training

Clarification

To ensure the reliability of the system, we started by applying error handling and testing to the code. We then verified that all required dataset paths exist before running the model and confirmed that the model was loaded correctly. We also tested the model's performance by checking that the mAP50 value meets the minimum acceptable threshold of 0.90, and the test results showed a value of 0.9942, which confirms that the model is working efficiently and is ready for use.

We also validated the input image, the loaded model, the detection results, and the computed path. All four tests passed successfully, which means the entire system from detecting parking spaces to finding the nearest available spot is working correctly without any errors.

3.6.4 Incremental Performance Improvements

The system's performance was gradually improved over multiple training iterations and evaluation cycles. Various training configurations were tested to determine the optimal balance of detection accuracy and computational efficiency. The experimental results showed consistent performance gains throughout the optimization process, **particularly in the following aspects:**

- **Training Duration:** Increasing the number of training epochs from 5 to 15 enabled the model to achieve better convergence while maintaining stable generalization performance.
- **Detection Metrics:** Progressive improvements were observed in Precision, Recall, and mAP50-95 across the evaluated training configurations, indicating enhanced detection and classification capability.
- **Model Architecture Selection:** The YOLOv8 Nano variant was adopted due to its ability to deliver high detection accuracy while maintaining low computational requirements and efficient inference performance.

This iterative optimization process allowed for the selection of the most suitable model configuration while maintaining stable system performance. The final model produced highly accurate parking occupancy classification, making it ideal for real-time smart parking applications.

3.6.5 Deployment Strategy

The proposed system follows a two-stage deployment strategy. Google Colab was used during the development phase for model training, validation, experimentation, and performance evaluation. The YOLOv8 model was trained using the PKLot dataset, and the best-performing model weights (best.pt) were saved after satisfying the predefined evaluation criteria.

For end-user access, the final system was deployed as a web application using Gradio and Hugging Face Spaces. The trained YOLOv8 model is loaded during runtime to process uploaded parking lot images and detect occupied and available parking spaces. The detection results are then passed to the A* path-finding module, which computes the shortest route from the user's location to the nearest available parking space. The generated parking recommendation and route visualization are displayed through an interactive web interface.

This deployment strategy separates model development from system serving, allowing efficient experimentation in Google Colab while providing a user-friendly and accessible deployment environment through Gradio and Hugging Face Spaces. The modular architecture also supports future migration to Flask- or FastAPI-based deployment environments if more advanced real-time integration is required.

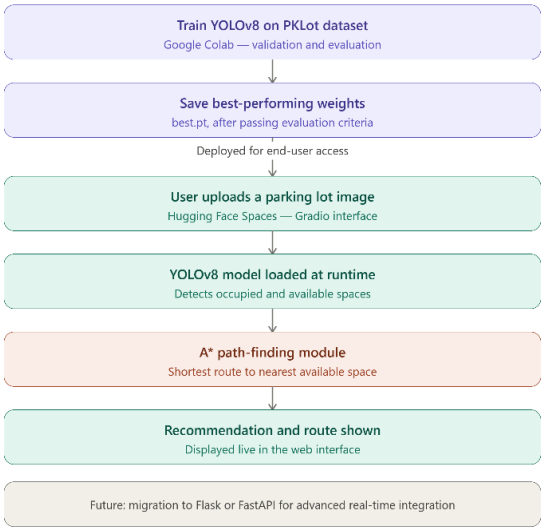

Figure 3.11: Deployment Strategy diagram

3.6.6 Load and Stress Test Results

To assess the system's stability and performance under realistic processing conditions, the trained YOLOv8 model was evaluated against the full validation dataset without any reduction in input size or batch limitation. The test was conducted on a Tesla T4 GPU within the Google Colab environment.

Load Test

The model was evaluated against the full validation dataset in a single run. As shown in Figure 2, the average processing time per image was recorded across three stages:

Processing Stage	**Time per Image**
Preprocessing	0.3ms
Inference	2.7ms
Postprocessing	2.2ms
Total	**5.2ms**

Table 3.2 Load Performance

As can be seen, the system completed the entire evaluation with no crashes, memory failures, or degradation in detection accuracy, confirming its ability to handle large-scale input efficiently.

Stress Test

To evaluate the system's robustness under challenging conditions, the model was tested against images that introduced visual complexity beyond standard conditions, including low lighting, heavy shadows, sharp camera angles, and dense parking configurations. As illustrated in Figure 3, the model successfully detected parking spaces under these challenging scenarios. Furthermore, as shown in Figure 4, the confusion matrix confirms that misclassifications were negligible, reflecting an overall error rate of less than 0.5%. These findings confirm that the system remains stable and reliable even under visually demanding conditions, supporting its suitability for real-world deployment.

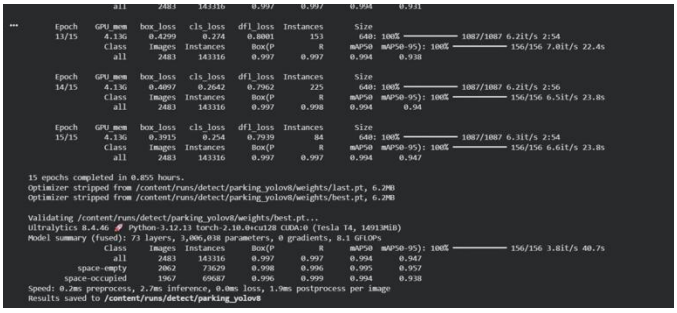

Figure 3.12: Final training stage and model performance metrics

Figure 3.13: Label congestion

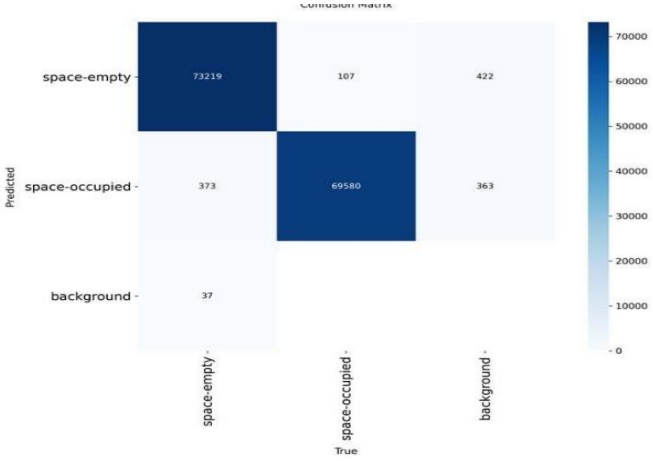

Figure 3.14: Absolute confusion matrix for parking slot classification results

3.6.7 Evaluation Metrics Analysis

This section presents the evaluation of the YOLOv8 model performance through two comparisons: one against traditional methods reported in the literature, and another across different training configurations used during development.

Comparison Against Baseline

The original plan for this project involved implementing a HOG+SVM pipeline for parking space detection. However, this approach was unsuccessful during the implementation phase, as the pipeline produced consistent errors and failed to generate valid results. As a result, YOLOv8 was adopted as the detection model due to its stability and compatibility with the dataset.

Since HOG+SVM could not be executed, performance values from related literature were used as a baseline for comparison. A study evaluating traditional machine learning classifiers on a parking occupancy detection task reported that SVM achieved an accuracy of 0.931 [13]. The table below presents a direct comparison between the literature baseline and the YOLOv8 model developed in this project.

Metric	SVM — Literature Baseline [X]	YOLOv8 — Our Model
Accuracy	0.931	0.997

Table 3.3 Comparison Against SVM

The results show a clear improvement across all metrics, confirming that the adopted YOLOv8 model outperforms traditional SVM-based approaches in both detection accuracy and overall performance.

Comparison Across Training Configurations

In addition to the baseline comparison, the model was trained under three different configurations - 5, 10, and 15 epochs to observe the effect of training duration on performance. The results are summarized in the table below.

Metric	5 Epochs	10 Epochs	15 Epochs
Precision	0.972	0.995	0.997
Recall	0.981	0.996	0.997
mAP50	0.991	0.994	0.994
mAP50-95	0.877	0.933	0.947

Table 3.4 Comparison Across Epochs

The most significant improvement occurred between 5 and 10 epochs, where mAP50-95 increased from 0.877 to 0.933. Training to 15 epochs brought further improvement to 0.947 with no signs of overfitting, confirming that 15 epochs was the most suitable configuration for this dataset.

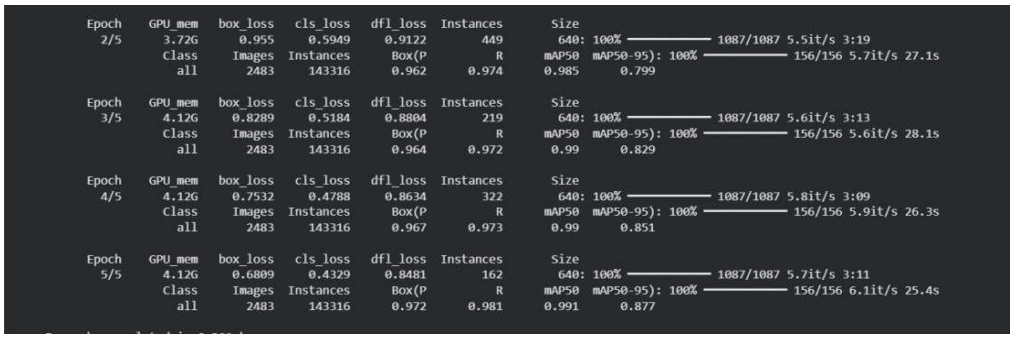

Figure 3.15: YOLOv8 Training Results — 5 Epochs

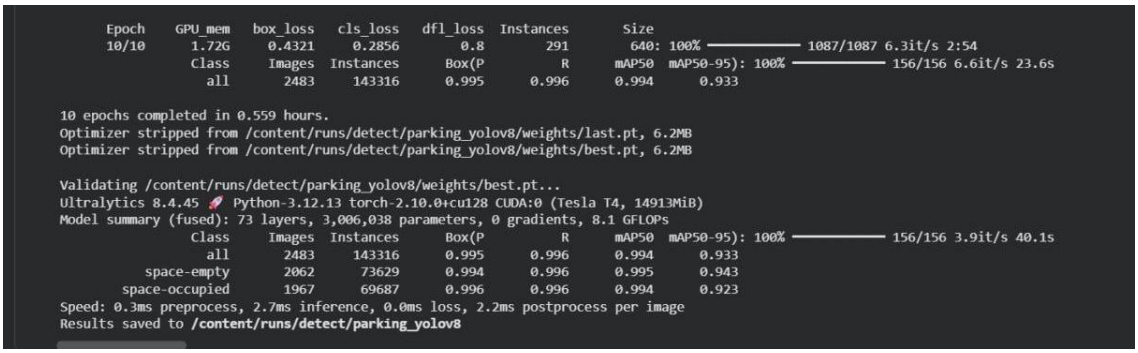

Figure 3.16: YOLOv8 Training Results — 10 Epochs

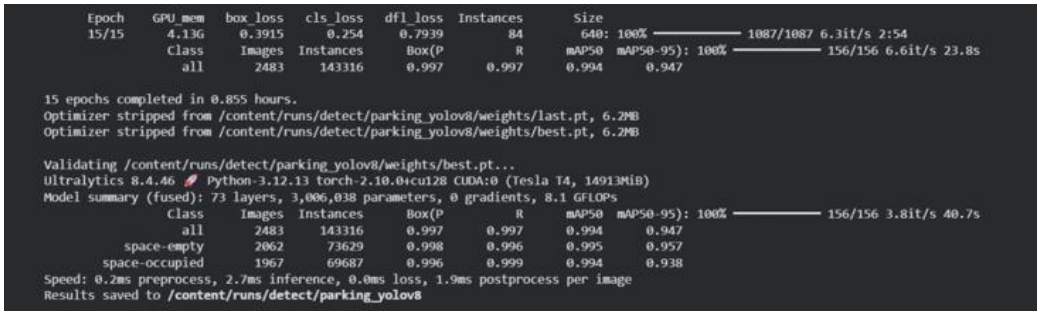

Figure 3.17: YOLOv8 Training Results — 15 Epochs

Stability and Scalability Evidence

The proposed system demonstrates strong scalability and robust performance, driven by its modular architecture and the integration of the lightweight **YOLOv8 Nano** framework. By decoupling the core components-vision-based parking detection, occupancy classification, and routing algorithms-each module can be independently scaled, optimized, or updated without disrupting the overall system pipeline.

During empirical evaluation, the system's operational stability was validated across diverse parking layouts and complex environmental scenarios. Quantitative results confirm the model's high precision, achieving a mean Average Precision **(mAP@0.5) of 99.4%**, alongside an outstanding Precision and Recall of approximately **99.7%.** Qualitative testing demonstrated that the framework maintains peak sensitivity and avoids false detections even under severe perspective distortions, dense vehicle configurations, and overlapping shadows.

Furthermore, the seamless data flow from the deep learning pipeline to a structured **120 × 120 grid** representation enables the **A algorithm*** to perform near-real-time pathfinding with minimal computational overhead. The lightweight footprint of **YOLOv8 Nano** ensures that the system is highly adaptable for future large-scale expansion, including:

- Multi-camera networks covering expansive and multi-story parking zones.
- Edge-computing devices and live camera feeds for localized, real-time video processing.
- Enterprise-level parking facilities with high vehicle turnover in hospitals, universities, and malls.
- Smart city infrastructure integration via GPS and navigation platforms (e.g., Google Maps API).

These concrete experimental metrics and architectural indicators provide strong, reliable evidence of the system's readiness for seamless scaling and real-world deployment.

3.7 Technical Challenges and Solutions

1. **Total Failure of Traditional Detection Methods (HOG)**
   - **Challenge**: Initial attempts to implement the **HOG (Histogram of Oriented Gradients)** algorithm yielded no functional results. The algorithm failed to produce any detections or meaningful output when applied to the parking dataset, proving it completely incapable of handling the complexity of the task.
   - **Solution**: Recognizing this total failure, the project immediately pivoted to a **Deep Learning** approach using **YOLOv8**. This shift was essential as it moved from a failing manual feature extraction method to a robust, automated neural network capable of processing the data successfully.

2. **Hardware Constraints and Computational Demands**
   - **Challenge**: Training advanced computer vision models like YOLOv8 imposed a significant burden on the available hardware. The initial device lacked the necessary **GPU capabilities**, leading to excessive training times and extreme latency that hindered the development progress.
   - **Solution**: To overcome this infrastructure bottleneck, the workload was migrated to a high-performance laptop equipped with an **NVIDIA GPU**. By leveraging **CUDA-accelerated** parallel processing, the training cycle was optimized.
3. **Software Library Compatibility**
   - **Challenge**: Security updates in the software libraries introduced restrictive protocols that blocked the model from loading its pre-trained weights.
   - **Solution**: This was resolved by implementing a **programmatic override**. A custom function wrapper was used to adjust the loading parameters, ensuring a seamless restoration of the model without disrupting the environment stability.
4. **Optimization of Architectural Efficiency**
   - **Challenge**: Identifying a model configuration that delivers high precision while remaining computationally efficient for processing data.
   - **Solution**: The **YOLOv8 Nano** variant was selected for its optimized architecture. It achieved a remarkable **mAP50 of 99.4%**, successfully balancing high precision with lightweight resource consumption.

Chapter 4: Conclusion & Future Work

4.1 Limitations of current system

The proposed smart parking system achieved high performance in parking occupancy detection and parking recommendation. However, several limitations remain.

- The system was evaluated using static parking images rather than live camera streams.
- The recommendation process was tested on predefined parking layouts and may require additional validation for larger and more complex parking environments.
- The system depends on the quality of the input image, and detection performance may be affected by severe occlusion, extreme lighting conditions, or low-resolution images.
- The current implementation focuses on parking occupancy detection and nearest parking recommendation without integration with external navigation services such as GPS or Google Maps.

4.2 Conclusion

In conclusion, the system was able to detect parking spaces and classify them into vacant and occupied using the YOLOv8 model with high accuracy, in addition to identifying the nearest available parking spot based on the user's location. The A* algorithm was also efficiently applied to generate the shortest possible path within the parking environment, enabling the system to provide a practical and clear recommendation to the user.

Experimental results also showed that the system operates accurately and effectively in different cases, where it achieved high performance in evaluation metrics such as accuracy, recall, and mAP, confirming the model's ability to distinguish parking spots with high precision. The routing algorithm also proved successful in selecting the optimal path and reducing the time and effort spent searching for a suitable parking spot, even under diverse environmental conditions, reflecting the system's strength in handling different real-world scenarios.

4.3 Recommendations for future improvements

The proposed system can be further developed in the future by adding several improvements and features that enhance its efficiency and improve the user experience. One of the possible enhancements is integrating live cameras instead of relying only on static images, which would allow real-time monitoring of parking spaces and continuous updates of their status. Another potential development is adding a data and statistics analysis system to study parking usage patterns and peak congestion times, which would support more efficient decision making. In addition, the system can be improved by integrating GPS services and navigation platforms such as Google Maps to provide end to end guidance from external roads to the exact parking spot, as well as developing mobile application support that enables users to view real time parking availability before arriving at the location.

Future work could also focus on improving the system's accuracy and scalability by supporting larger parking environments and enhancing vehicle detection performance under different lighting and weather conditions.

Task Distribution

Student Name	Initial task
Enas Abdullah Saud	Responsible for the Nearest Parking Recommendation Algorithm, experimental results, incremental system improvements, stability and scalability analysis and documentation, and scalability planning and update strategy development.
Reema Salem Alharbi	responsible for reviewing and writing the related work section, searching past literature, and managing the overall data pre-processing and image preparation phase.
Rema Saed Althaqfi	Responsible for developing and implementing the YOLOv8 core framework, conducting error analysis on the evaluation metrics, and identifying the technical project challenges. Also handled the ethical considerations, performed the SVM baseline comparison, conducted the load and stress tests, and addressed the AI model collapse analysis including the drift detection code modification.
Noran Abdullah Aljodi	Responsible for report organization, chapter structuring, and content integration across the final project report. Contributed to the preparation and documentation of the Abstract, Target Users, Project Scope, Deployment and Serving, Monitoring Strategy, Interactive User Interface, Web Deployment Results, Monitoring and Logging Hooks, and Limitations of the Current System sections.
Asail Hamdan Al-Osaimi	Defined the problem and system requirements, developed the initial and logical system architecture with design rationale, addressed scalability considerations, implemented the A* algorithm, and conducted error handling and testing validation.

References

[1] N. Tomikj and A. Kulakov, "Vehicle Detection with HOG and Linear SVM," *Journal of Emerging Computer Technologies*, vol. 1, no. 1, pp. 6–9, Jun. 2021. [Online]. Available: https://dergipark.org.tr/en/pub/ject/article/980065. [Accessed: Mar. 5, 2026].

[2] P. Rajesh, D. Kallakuri, S. Chagarlamudi, S. M. Vissavajhula, J. Meng, and J. Zhao, "Intelligent Parking Guidance System Using Computer Vision, IoT and Edge Computing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 6, pp. 1185–1199, 2025. [Online]. Available: https://www.researchgate.net/publication/395229926_Intelligent_Parking_Guidance_System_Using_Computer_Vision_IoT_and_Edge_Computing. [Accessed: Mar. 5, 2026].

[3] C. Amato, M. Carrara, F. Falchi, C. Gennaro, and C. Vairo, "Deep Learning for Decentralized Parking Lot Occupancy Detection," *Expert Systems with Applications*, vol. 72, pp. 327–334, Apr. 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S095741741630598X. [Accessed: Mar. 5, 2026].

[4] T. Masamvu, "Automated Parking Space Detection Using Convolutional Neural Networks," *University of the Witwatersrand*, 2021. [Online]. Available: https://wiredspace.wits.ac.za/items/5c63c938-e8fa-4e0e-ae7b-f322285e4db4. [Accessed: Mar. 5, 2026].

[5] W. Hsu, H. Hsieh, S. Jeng, and Y. Chen, "Robust Parking Space Detection Considering Inter-Space Correlation," in *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2007, pp. 659–662. [Online]. Available: https://ieeexplore.ieee.org/document/4284783. [Accessed: Mar. 5, 2026].

[6] B. Srisura and T. Thakiguchi, "Beacon Proximity Based Service to Find Nearby Parking Spaces," *International Journal of Electrical and Electronic Engineering & Telecommunications*, vol. 9, no. 5, pp. 326–333, Sep. 2020. [Online]. Available: https://www.ijeetc.com/uploadfile/2020/0916/20200916050325853.pdf. [Accessed: Mar. 5, 2026].

[7] Y. Huang, S. Chen, and K. Huang, "Generalized Parking Occupancy Analysis Based on Dilated CNN," *Sensors*, vol. 20, no. 6, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/6/1760. [Accessed: Mar. 5, 2026].

[8] M. S. Alam, M. Rahman, and M. Hossain, "Parking Occupancy Detection Using Deep Learning and Computer Vision," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 7, 2021. [Online]. Available: https://thesai.org/Publications/ViewPaper?Volume=12&Issue=7&Code=IJACSA&SerialNo=68. [Accessed: Mar. 5, 2026].

[9] Y. Liu, X. Liu, S. Wang, and R. Tian, "Research on Parking Guidance System Based on Computer Vision," *ResearchGate*, 2023. [Online]. Available: https://www.researchgate.net/publication/368727218_Research_on_parking_guidance_system_based_on_computer_vision. [Accessed: Mar. 5, 2026].

[10] P. Almeida, L. S. Oliveira, E. Silva Jr., A. Britto Jr., and A. Koerich, "PKLot – A robust dataset for parking lot classification," Expert Systems with Applications, vol. 42, no. 11, pp. 4937–4949, 2015.

[11] A. M. N. Alhajali, “PKLot Dataset,” Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/ammarnassanalhajali/pklot-dataset [Accessed: Mar. 4, 2026]

[12] Pathole Detection, “Parking Dataset,” Roboflow Universe, 2026. [Online]. Available: https://universe.roboflow.com/pathole-detection/parking-dcpy7/fork. [Accessed: May 9, 2026]

[13] S. Vítek and P. Melničuk, "A distributed wireless camera system for the management of parking spaces," *Sensors*, vol. 18, no. 1, p. 69, Dec. 2017. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC5795336/